

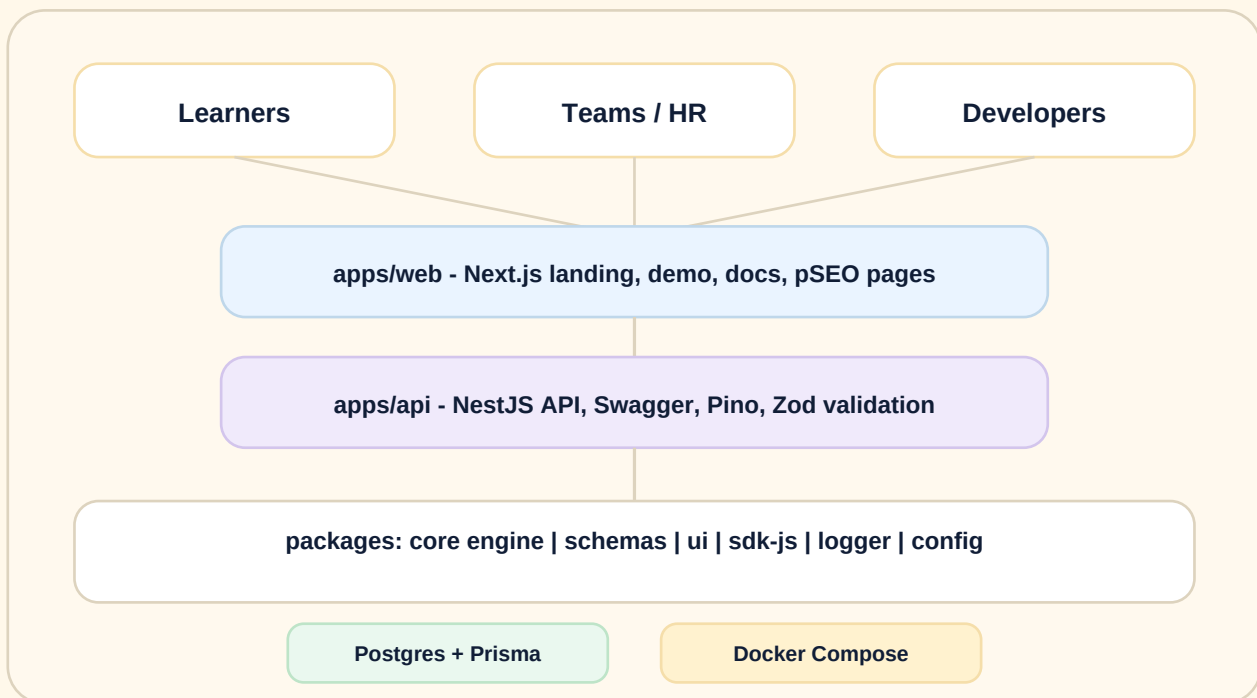
LearnKit AI

Initial Architecture & Coding Agent Handoff

Repository: github.com/learnkit-ai/learnkit

Domain: learnkit-ai.com

Product: open-source infrastructure for personalized AI learning paths



Recommended first milestone: create a production-quality monorepo foundation, implement the landing page and mock interactive demo, add a NestJS API skeleton, shared schemas, Docker, CI, agent instructions, and open-source documentation. Do not build a real AI backend in the first PR.

1. Executive Summary

LearnKit AI is open-source infrastructure for creating personalized AI learning paths from a user's role, tools, goals, skill level, and optional company knowledge. The project should start as a serious open-source monorepo that coding agents can extend safely.

The first implementation should prove the product experience with deterministic mock data: a user chooses a role, tools, and goal; the app generates a role-specific learning path; the user can preview lessons and see a contextual AI Guide note.

- **Frontend:** high-quality Next.js landing page, interactive demo, developers page, teams page, docs placeholder, and SEO-friendly content routes.
- **Backend:** NestJS skeleton with typed endpoints, Zod validation, Pino logging, Swagger/OpenAPI, and mock responses.
- **Shared packages:** schemas, core deterministic generator, UI primitives, JavaScript SDK placeholder, config, logger.
- **Open-source readiness:** README, CONTRIBUTING, ROADMAP, SECURITY, CODE_OF_CONDUCT, LICENSE, AGENTS.md, CLAUDE.md, CI, Docker, Dependabot.

2. Product Context

Audience	Question they need answered	Product answer
Individual learners	How will this help my job today?	Generate short role-specific lessons based on their tools and goals.
Teams / HR / L&D	Can this scale across teams and prove learning impact?	Showing analytics, progress, skill growth, completion, and practical evaluation.
Developers / platform teams	Can I integrate this into my app or internal training system?	Offering SDKs and infrastructure for learning path and lesson generation.

Brand Rules

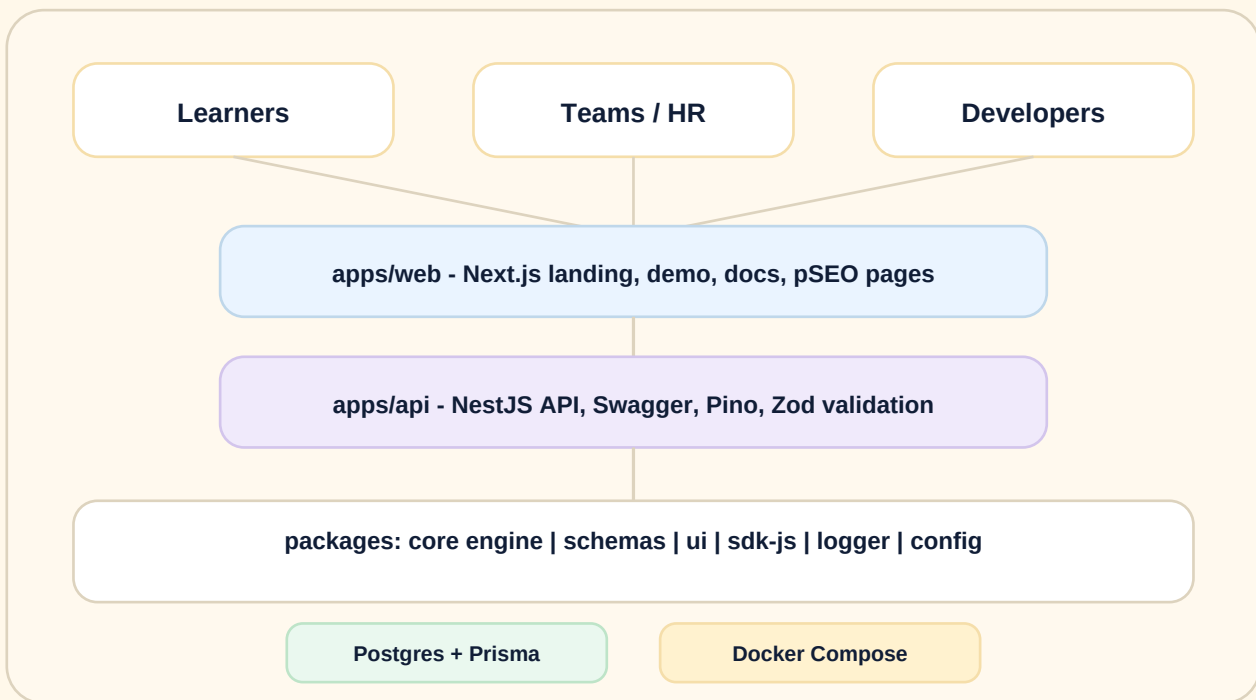
- Public product name: **LearnKit AI**. Do not use only "LearnKit" in public-facing UI.
- Domain: **learnkit-ai.com**. API: **api.learnkit-ai.com**. Docs: **docs.learnkit-ai.com**.
- GitHub: **github.com/learnkit-ai/learnkit**. NPM scope: **@learnkit-ai**.
- Do not use Scholé branding, Scholé copy, or the Olé tutor name. Use **AI Guide** or **LearnKit Guide**.
- Do not include fake social proof: no fake logos, fake testimonials, fake SOC 2, fake GitHub stars, or fake enterprise metrics.

3. Architecture Decision

Decision: pnpm + Turborepo monorepo with Next.js frontend and NestJS backend.

Layer	Choice	Reason
Package manager	pnpm workspaces	Fast installs, strict dependency graph, strong monorepo support.
Build orchestration	Turborepo	Efficient task caching and orchestration across apps and packages.
Frontend	Next.js App Router	Great for marketing pages, docs, SEO routes, and app-like demo flows.
Backend	NestJS	Structured TypeScript API with modules, controllers, services, testing, and OpenAPI support.
Database	Postgres + Prisma	Portable open-source default with typed schema and migrations.
Validation	Zod	Shared validation between frontend, backend, and SDK.
Logger	Pino	Fast structured logging for Node and NestJS.
Testing	Vitest + Jest + Playwright	Vitest for packages/frontend utilities, Jest for NestJS, Playwright for E2E.
License	Apache-2.0	Open-source friendly with explicit patent grant.

Architecture Diagram



4. Monorepo Folder Structure

```
learnkit/
  .agent/
    agent.md
    settings.json
    commands/
      review.md
      fix-issue.md
      ship.md
    rules/
      code-style.md
      testing.md
      security.md
      api-conventions.md
    agents/
      code-reviewer.md
      security-auditor.md

  .github/
    workflows/
      ci.yml
      security.yml
    ISSUE_TEMPLATE/
      dependabot.yml
    pull_request_template.md
    copilot-instructions.md

  apps/
    web/      # Next.js marketing site, demo app, pSEO pages
    api/      # NestJS REST API, Swagger, Pino, Zod validation
    docs/     # Documentation site or placeholder route/app

  packages/
    core/     # deterministic learning path and lesson engine
    schemas/  # Zod schemas and inferred TypeScript types
    ui/       # reusable UI primitives
    sdk-js/   # JavaScript SDK placeholder
    config/   # shared ESLint, Prettier, Tailwind, tsconfig
    logger/   # Pino wrapper

  examples/
    nextjs-embed/
    api-client/
    self-hosted/

  docker/
    web.Dockerfile
    api.Dockerfile

  prisma/
    schema.prisma
    seed.ts

  docker-compose.yml
  pnpm-workspace.yaml
  turbo.json
  tsconfig.base.json
  package.json
  README.md
  CONTRIBUTING.md
  ROADMAP.md
  SECURITY.md
  CODE_OF_CONDUCT.md
  LICENSE
  .env.example
  AGENTS.md
  CLAUDE.md
```

5. Applications

apps/web - Next.js

The web app should launch with the landing page, demo route, developers route, teams route, docs placeholder, and content-driven SEO pages. It should be mobile-first and use design tokens in CSS variables.

```
Routes:
/
/demo
/developers
/teams
/docs
/roles
/roles/[role]
/tools
/tools/[tool]
/use-cases
/use-cases/[slug]
/templates
/templates/[slug]
```

Homepage Sections

- Navbar with LearnKit AI logo, Demo, Teams, Developers, Docs, GitHub, and primary CTA.
- Hero with live product preview showing role, tools, goal, generated path, and AI Guide feedback.
- Audience paths: Learners, Teams / HR / L&D;, Developers / API.
- Interactive demo preview with role -> tools -> goal -> curriculum -> lesson preview.
- Open-source section with honest repo-first positioning and GitHub CTA.
- Developer API section with request/response preview and docs CTA.
- Teams / HR section with analytics mock and practical progress tracking.
- Pricing / availability: Open Source free self-hosted, Cloud coming soon, Teams/API request access.
- FAQ, final CTA, and footer.

apps/api - NestJS

The API should be structured as if it will become real, even if v0 returns mock data. Use module boundaries from the start.

```
Modules:
health
learning-paths
lessons
api-access
team-demo

Endpoints:
GET  /health
POST /v1/learning-paths
GET  /v1/learning-paths/:id
POST /v1/lessons
POST /v1/api-access-requests
POST /v1/team-demo-requests
```

- Use Zod schemas from packages/schemas for request validation.
- Use Pino for structured request logging.
- Add Swagger/OpenAPI setup for API documentation.
- Return mock data initially, with service boundaries ready for Prisma and LLM integrations later.

6. Shared Packages

Package	Purpose	Initial contents
packages/schemas	Shared Zod validation and inferred types	LearningPathInput, Lesson, ApiAccessRequest, TeamDemoRequest, ProgressEvent
packages/core	Deterministic learning path engine	generateLearningPath, generateLesson, fixtures, prompt templates later
packages/ui	Shared UI primitives	Button, Card, Badge, Chip, Input, Tabs, Accordion
packages/sdk-js	Future typed JavaScript SDK	createLearningPath, getLearningPath, createLesson, requestApiAccess
packages/logger	Shared Pino wrapper	createLogger, request logging helpers
packages/config	Shared repo configs	ESLint, Prettier, Tailwind, TypeScript configs

7. Interactive Demo Specification

The demo is the core product moment. It should be deterministic and backend-free for v0, but structured so real API calls can replace mock functions later.

Step	Behavior
1. Role	User selects Marketing, Finance, Data, Design, Operations, HR, Product, or Engineering.
2. Tools	User selects tools such as Excel, Notion, Slack, Google Docs, HubSpot, Figma, Python, Cursor, GitHub, Linear.
3. Goal	User enters or selects a practical goal.
4. Generation	Animated phases: Reading your role, Mapping tools to tasks, Sequencing lessons, Creating practice work, Preparing your
5. Curriculum	Show generated path, 4-week/30-day plan, lesson cards, duration, difficulty, why, practice, outcome.
6. Lesson preview	Clicked lesson opens detail panel with contextual AI Guide note based on role/tool/goal.

Mock API Functions

/lib/mock-api.ts

```
export async function generateLearningPath(input: LearningPathInput):  
  Promise<LearningPath>  
export async function generateLesson(pathId: string, lessonId: string):  
  Promise<Lesson>  
export async function trackProgress(event: ProgressEvent): Promise<{ ok: true }>  
export async function requestApiAccess(payload: ApiAccessRequest): Promise<{ ok: true  
  }>  
export async function bookTeamDemo(payload: TeamDemoRequest): Promise<{ ok: true }>
```

8. TypeScript and Zod Models

```
LearningPathInput:
  role: string
  tools: string[]
  goal: string
  level: 'beginner' | 'intermediate' | 'advanced'

LearningPath:
  id: string
  title: string
  summary: string
  durationDays: number
  lessons: Lesson[]
  outcomes: string[]

Lesson:
  id: string
  title: string
  duration: string
  difficulty: 'Beginner' | 'Intermediate' | 'Advanced'
  why: string
  practice: string
  expectedOutcome: string
  tools: string[]
  role: string

ProgressEvent:
  lessonId: string
  status: 'started' | 'completed' | 'skipped'
  timestamp: string

ApiAccessRequest:
  name: string
  email: string
  company?: string
  useCase: string

TeamDemoRequest:
  name: string
  email: string
  company: string
  teamSize?: string
  message?: string
```

9. Prisma Data Model - Initial Draft

Keep the database optional for v0. The app should run with mock data, but the schema should establish future direction.

```
Models:
User
  id, email, name, createdAt, updatedAt

LearningPath
  id, userId optional, role, tools Json, goal, level, title, summary, durationDays,
  createdAt, updatedAt

Lesson
  id, learningPathId, title, duration, difficulty, why, practice,
  expectedOutcome, tools Json, role, order, createdAt, updatedAt

ProgressEvent
  id, userId optional, lessonId, status, metadata Json, createdAt

ApiAccessRequest
  id, name, email, company, useCase, status, createdAt

TeamDemoRequest
  id, name, email, company, teamSize, message, status, createdAt
```

10. Developer API Preview

Display this API shape in the developer page and docs. Use mock backend responses until the real engine is wired.

POST <https://api.learnkit-ai.com/v1/learning-paths>

Request:

```
{
  "role": "Marketing Manager",
  "tools": ["HubSpot", "Google Docs", "Slack"],
  "goal": "Use AI to improve campaign planning",
  "level": "beginner"
}
```

Response:

```
{
  "path": "AI for Campaign Strategy",
  "lessons": [
    {
      "title": "Use AI to analyze audience segments",
      "duration": "8 min",
      "difficulty": "Beginner",
      "practice": "Use AI to identify 3 audience segments from a campaign brief."
    },
    {
      "title": "Generate campaign briefs faster",
      "duration": "10 min",
      "difficulty": "Beginner",
      "practice": "Turn a rough campaign idea into a structured brief."
    }
  ]
}
```

11. SEO and pSEO Structure

The SEO strategy should be content-driven and high-quality. Avoid spammy mass-generated pages. Each page should answer a real search intent and push users toward the demo, GitHub, or API access.

Route family	Example pages	Purpose
/roles/[role]	/roles/marketing-manager, /roles/product-manager	Share role-specific learning path examples and demo CTA.
/tools/[tool]	/tools/hubspot, /tools/slack, /tools/cursor	Explain how LearnKit AI adapts learning to tools people already use.
/use-cases/[slug]	/use-cases/team-ai-training, /use-cases/ai-ops	Capturing practical business use cases.
/templates/[slug]	/templates/marketing-ai-learning-path	Offer starter templates for learning paths.

- Every content page needs metadata title, description, canonical URL, readable content, and CTA.
- Use static content files in apps/web/content for seed pages.
- Add structured FAQ content where appropriate.
- No fake statistics or fake proof on SEO pages.

12. Quality, Testing, and Security

Area	Decision
Lint and format	ESLint + Prettier at root, shared config in packages/config.
Type checking	TypeScript strict mode where reasonable, root turbo typecheck.
Unit tests	Vitest for shared packages and frontend utilities. Jest for NestJS.
E2E tests	Playwright for homepage and demo flow.
Security docs	SECURITY.md with vulnerability reporting and secret handling.
Dependency hygiene	Dependabot for pnpm/npm, GitHub Actions, Docker.
Code scanning	GitHub CodeQL workflow where practical.
Runtime hardening	Helmet, CORS config, rate-limit placeholder, Zod validation, Pino logs.
Accessibility	Semantic HTML, keyboard navigation, focus states, reduced motion support, contrast checks.

Root Scripts

```
{
  "dev": "turbo dev",
  "build": "turbo build",
  "lint": "turbo lint",
  "typecheck": "turbo typecheck",
  "test": "turbo test",
  "test:e2e": "turbo test:e2e",
  "format": "prettier --write .",
  "format:check": "prettier --check ."
}
```

13. Docker and Hosting

The project should support both hosted deployment and self-hosted open-source usage. The initial Docker setup should be simple and reliable.

```
docker-compose.yml services:
  web      # Next.js production build
  api      # NestJS API
  postgres # Postgres database
  redis    # optional placeholder for future queues/cache
```

Hosting path	Recommended services	When to use
Fastest launch	Vercel + Railway + Neon	Best for quick public preview and iteration.
All-in-one simple	Railway	Good when keeping web, api, db close together.
Serious infra	Fly.io + Neon + Upstash	Good when needing global app hosting and managed data services.
Self-hosted OSS	Docker Compose on VPS	Required for credible open-source adoption.

14. Agent Instructions Strategy

Use a universal .agent/agent.md as the source of truth, then mirror core instructions into AGENTS.md for Codex, CLAUDE.md for Claude Code, and optionally .github/copilot-instructions.md.

```
.agent/agent.md      # universal source of truth
AGENTS.md           # Codex reads this
CLAUDE.md           # Claude Code reads this
.github/copilot-instructions.md
```

Core Agent Rules

- Use LearnKit AI as the public brand.
- Do not use Scholé or Olé references.
- Do not invent social proof.
- Keep demo data separate from UI.
- Use shared Zod schemas in frontend and backend.
- Run lint, typecheck, test, and build before considering work complete.
- Prefer small reusable components and clear module boundaries.

15. First PR Scope

Do not build a real AI backend in PR 1. Build the repo foundation and product demo that proves the direction.

- Set up pnpm, Turborepo, TypeScript, Tailwind, Next.js, NestJS, shared packages.
- Implement homepage, /demo, /developers, /teams, /docs, and seed SEO routes.
- Implement deterministic interactive demo using mock data and mock API functions.
- Create NestJS API skeleton with health and v1 endpoints returning mock data.
- Create shared schemas, core generator package, UI primitives, SDK placeholder, logger package.
- Add Dockerfiles, docker-compose, .env.example, Prisma schema, and seed placeholder.
- Add README, CONTRIBUTING, ROADMAP, SECURITY, CODE_OF_CONDUCT, LICENSE, AGENTS.md, CLAUDE.md.
- Add CI, security workflow, Dependabot, PR template, and basic tests.

16. Acceptance Criteria

- pnpm install works.
- pnpm dev works.
- pnpm lint passes.
- pnpm typecheck passes.
- pnpm test passes.
- pnpm build passes.
- Docker Compose starts web, api, and postgres.
- Homepage is responsive and communicates the product within 10 seconds.
- Demo flow works with mock data and no backend dependency.
- API health endpoint works.
- Shared Zod schemas are used by API and frontend or are ready to be used by both.
- AGENTS.md and CLAUDE.md exist.
- README, CONTRIBUTING, ROADMAP, SECURITY, CODE_OF_CONDUCT, LICENSE exist.
- No Scholé or Olé references remain.
- No fake proof/social proof is included.
- Code is componentized and easy to extend.

17. Coding Agent Master Prompt

Paste the following into Codex, Claude Code, or another coding agent after connecting the repository.

You are working on <https://github.com/learnkit-ai/learnkit>.

Project: LearnKit AI
Domain: learnkit-ai.com
GitHub org: [learnkit-ai](https://github.com/learnkit-ai)
Repo: [learnkit](https://github.com/learnkit-ai/learnkit)

Goal:

Set up LearnKit AI as a production-quality open-source monorepo.
LearnKit AI is open-source infrastructure for generating personalized AI learning paths from a user's role, tools, goals, skill level, and optional company knowledge.

Use this stack:

- pnpm workspaces
- Turborepo
- Next.js App Router for apps/web
- NestJS for apps/api
- TypeScript
- Tailwind CSS
- Zod for shared validation
- Prisma + Postgres schema setup
- Pino for logging
- Vitest, Jest, Playwright
- Docker + docker-compose
- ESLint + Prettier
- GitHub Actions CI
- Dependabot
- Apache-2.0 license

Brand rules:

- Use LearnKit AI everywhere public-facing.
- Use learnkit-ai.com, api.learnkit-ai.com, docs.learnkit-ai.com.
- Do not use Scholé branding or Olé. Use AI Guide or LearnKit Guide.
- Do not include fake customer logos, fake testimonials, fake SOC 2, fake GitHub stars, or fake enterprise metrics.

Build:

- apps/web with /, /demo, /developers, /teams, /docs, /roles, /tools, /use-cases, /templates routes.
- apps/api with health, learning-paths, lessons, api-access, team-demo modules.
- packages/core, packages/schemas, packages/ui, packages/sdk-js, packages/logger, packages/config.
- Dockerfiles, docker-compose, Prisma schema, CI, agent docs, open-source docs.

First PR acceptance:

- pnpm install/dev/lint/typecheck/test/build all work.
- Docker Compose starts web/api/postgres.
- Homepage is responsive.
- Demo flow generates a mock path.
- API health endpoint works.
- No fake proof, no Scholé, no Olé.
- Code is componentized and ready to extend.

18. AGENTS.md Draft

AGENTS.md

Project

LearnKit AI is open-source infrastructure for creating personalized AI learning paths from a user's role, tools, goals, skill level, and optional company knowledge.

Repository: github.com/learnkit-ai/learnkit
Domain: learnkit-ai.com
Public product name: LearnKit AI

Brand rules

Use "LearnKit AI" in public-facing UI. Do not use only "LearnKit" as the public brand.

Do not use Scholé branding, Scholé copy, or the Olé tutor name. Use "AI Guide" or "LearnKit Guide" instead.

Do not create fake social proof: no fake customer logos, fake testimonials, fake GitHub stars, fake SOC 2 claims, or fake enterprise metrics.

Tech stack

- Next.js App Router
- NestJS
- React
- TypeScript
- Tailwind CSS
- Zod
- Prisma + Postgres
- Pino
- pnpm + Turborepo

Code quality

Prefer small reusable components. Keep demo data separate from UI. Keep mock API functions separate. Use TypeScript types and Zod schemas. Do not add unnecessary dependencies. Ensure responsive behavior works on mobile, tablet, and desktop.

Commands

```
pnpm dev
pnpm lint
pnpm typecheck
pnpm test
pnpm build
```

UX principles

Every major section should answer: what does this do, why should I care, and what can I try next. Avoid vague AI buzzwords. Prefer practical, outcome-driven copy.

End of document.